```
/**
 * @file sdcard.c
 * @brief Implementação completa do Driver SD Card SPI + FAT32 (Root-only) para AVR
 * @version V1.0
 * @author Tiago Henrique dos Santos / Canal do Youtube: Ciência Elétrica
 * @date 2026
 */

#include "sdcard.h"

static SD_Status sd_expand_directory(SD_Context *ctx, FAT32_Info *fs, uint32_t start_cluster);

SD_Status sd_fat_get_next_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster, uint32_t *next_cluster);

/* ================================================================
   ESTRUTURA PARA DATA E HORA
   ================================================================ */
SD_DateTime g_sd_datetime = {2026, 3, 30, 10, 20, 0};
static uint16_t _sd_make_fat_time(uint8_t h, uint8_t m, uint8_t s);
static uint16_t _sd_make_fat_date(uint16_t year, uint8_t month, uint8_t day);

/* ================================================================
   FUNÇÕES ESTÁTICAS / INTERNAS (LOW LEVEL)
   ================================================================ */

/**
 * @brief Transmissão/recepção de um byte via SPI (função interna)
 *
 * Função estática usada por toda a biblioteca.
 * Bloqueia até a transmissão terminar (padrão AVR).
 */
static uint8_t _sd_spi_transfer(uint8_t data)
{
    SPDR = data;
    while (!(SPSR & (1 << SPIF)));
    return SPDR;
}

/**
 * @brief Ativa o Chip Select (CS) - Pino PB2
 *
 * Função estática para manter o HAL interno limpo.
 */
static void _sd_cs_low(void){SD_PORT &= ~SS_PIN;}

/**
 * @brief Desativa o Chip Select (CS) - Pino PB2
 *
 * Sempre deve ser chamado após qualquer comando para liberar o barramento.
 */
static void _sd_cs_high(void){SD_PORT |= SS_PIN;}

/**
 * @brief Envia um comando SPI para o cartão SD e aguarda resposta R1.
 *
 * @param ctx Ponteiro para o contexto da biblioteca.
 * @param cmd Número do comando (0-63).
 * @param arg Argumento de 32 bits específico do comando.
 * @param crc Byte de CRC (necessário apenas para CMD0 e CMD8).
 * @return SD_Status Status da resposta do cartão (R1).
 */
static SD_Status sd_cmd_send(SD_Context *ctx, uint8_t cmd, uint32_t arg, uint8_t crc)
{
    uint8_t r1;
    uint16_t timeout = 10000;
```

```
    /* Garante que o barramento esteja livre */
    ctx->hal.spi_tx_rx(0xFF);
    ctx->hal.cs_low();

    /* Envia o comando (bit 6 sempre setado) */
    ctx->hal.spi_tx_rx(0x40 | cmd);
    ctx->hal.spi_tx_rx((uint8_t)(arg >> 24));
    ctx->hal.spi_tx_rx((uint8_t)(arg >> 16));
    ctx->hal.spi_tx_rx((uint8_t)(arg >> 8));
    ctx->hal.spi_tx_rx((uint8_t)arg);
    ctx->hal.spi_tx_rx(crc);

    /* Aguarda resposta válida (R1) */
    do {
        r1 = ctx->hal.spi_tx_rx(0xFF);
    } while (r1 == 0xFF && --timeout);

    if (timeout == 0)
        return SD_ERR_TIMEOUT;

    return (SD_Status)r1;
}

/**
 * @brief Aguarda o cartão sair do estado de ocupado (busy).
 *
 * Monitora o pino MISO até que o cartão libere o barramento (0xFF).
 * O timeout é calculado dinamicamente com base na macro F_CPU para garantir
 * aproximadamente 250ms de espera, evitando travamentos em caso de falha física.
 *
 * @param ctx Ponteiro para o contexto da biblioteca.
 * @return SD_Status SD_SUCCESS se pronto, SD_ERR_TIMEOUT se falhar.
 */
static SD_Status sd_spi_wait_ready(SD_Context *ctx)
{
    uint32_t timeout = F_CPU / 64UL;   /* ~250 ms @ 16 MHz em velocidade FAST */
    uint8_t data;

    do {
        data = ctx->hal.spi_tx_rx(0xFF);
        if (data != 0x00)
            return SD_SUCCESS;
    } while (--timeout);

    return SD_ERR_TIMEOUT;
}

/* ================================================================
   INICIALIZAÇÃO
   ================================================================ */

/**
 * @brief Configura a velocidade do SPI no AVR (função interna)
 *
 * @param speed_mode SD_SPI_SLOW (fosc/64) ou SD_SPI_FAST (fosc/2 + SPI2X)
 * @note Usada automaticamente por sd_quick_init e sd_begin
 */
void sd_avr_spi_set_speed_default(uint8_t speed_mode)
{
    if (speed_mode == SD_SPI_SLOW) {
        /* Modo lento: fosc/64 */
        SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR1) | (1 << SPR0);
        SPSR &= ~(1 << SPI2X);
    } else {
        /* Modo rápido: fosc/2 */
        SPCR = (1 << SPE) | (1 << MSTR);
```

```
        SPSR |= (1 << SPI2X);
    }
}

/**
 * @brief Inicializa o cartão SD no modo SPI.
 *
 * Executa a sequência CMD0, CMD8 e ACMD41. Detecta automaticamente se o
 * cartão é SDSC (versão 1) ou SDHC/SDXC (versão 2+).
 *
 * @param ctx Ponteiro para o contexto da biblioteca.
 * @return SD_Status SD_SUCCESS em caso de sucesso ou código de erro.
 */
SD_Status sd_init(SD_Context *ctx)
{
    uint8_t i;
    SD_Status status = SD_ERR_TIMEOUT;

    /* 1. Configura velocidade inicial segura (lenta) */
    if (ctx->hal.set_speed != NULL) {
        ctx->hal.set_speed(SD_SPI_SLOW);
    } else {
        sd_avr_spi_set_speed_default(SD_SPI_SLOW);
    }

    ctx->hal.cs_high();

    /* 80 ciclos de clock para sincronização inicial */
    for (i = 0; i < 10; i++) {
        ctx->hal.spi_tx_rx(0xFF);
    }

    /* CMD0: Reset do cartão */
    if (sd_cmd_send(ctx, 0, 0, 0x95) != 0x01) {
        return SD_ERR_INITIALIZATION;
    }

    /* CMD8: Verificação de voltagem (melhorado para suportar SD v1) */
    uint8_t cmd8_response = sd_cmd_send(ctx, 8, 0x1AA, 0x87);
    if (cmd8_response == 0x01) {
        /* SD versão 2 ou superior ? descarta os 4 bytes da resposta R7 */
        for (i = 0; i < 4; i++) ctx->hal.spi_tx_rx(0xFF);
    } else if (cmd8_response == 0x05) {
        /* SD versão 1 ? comando ilegal é esperado, continua normalmente */
    } else {
        return SD_ERR_VOLTAGE;
    }

    /* ACMD41: Processo de inicialização do cartão */
    for (uint16_t retry = 0; retry < 0xFFFF; retry++) {
        sd_cmd_send(ctx, 55, 0, 0xFF);                  // APP_CMD
        if (sd_cmd_send(ctx, 41, 0x40000000, 0xFF) == 0x00) {
            status = SD_SUCCESS;
            break;
        }
    }

    if (status == SD_SUCCESS) {
        /* === DETECÇÃO DO TIPO DO CARTÃO (SDSC vs SDHC/SDXC) === */
        uint32_t ocr = 0;
        for (uint8_t j = 0; j < 4; j++) {
            ocr = (ocr << 8) | ctx->hal.spi_tx_rx(0xFF);
        }
        ctx->card_type = (ocr & (1UL << 30)) ? 2 : 1;   // Bit CCS

        /* Aumenta a velocidade do barramento para operação normal */
```

```
        if (ctx->hal.set_speed != NULL) {
            ctx->hal.set_speed(SD_SPI_FAST);
        } else {
            sd_avr_spi_set_speed_default(SD_SPI_FAST);
        }

        /* Fixa tamanho do bloco em 512 bytes (padrão) */
        sd_cmd_send(ctx, 16, 512, 0xFF);
    }

    ctx->hal.cs_high();
    ctx->hal.spi_tx_rx(0xFF);

    return status;
}

/**
 * @brief Tenta inicializar o cartão até 3 vezes com delay entre tentativas
 *
 * Função interna usada por sd_quick_init e sd_begin.
 * Útil quando o cartão demora para responder (ex: logo após inserção).
 */
SD_Status sd_init_retry(SD_Context *ctx)
{
    SD_Status s;

    for (uint8_t i = 0; i < 3; i++) {
        s = sd_init(ctx);
        if (s == SD_SUCCESS)
            return SD_SUCCESS;

        /* Mensagem de debug (opcional) */
        printf("Retry SD init (%u/3)\n", i + 1);
        ctx->hal.delay_ms(100);
    }

    return s;
}

SD_Status sd_quick_init(SD_Context *ctx)
{
    /* 1. Configuração física dos pinos SPI */
    SD_PORT |= SS_PIN;
    SD_DDR |= SS_PIN | MOSI_PIN | SCK_PIN;   // Saídas
    SD_DDR &= ~MISO_PIN;                     // MISO como entrada
    SD_PORT |= MISO_PIN;                     // Pull-up no MISO

    /* SPI Mestre, Modo 0, velocidade inicial lenta (fosc/64) */
    SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR1) | (1 << SPR0);

    /* 2. Injeta as funções internas no contexto HAL */
    ctx->hal.spi_tx_rx = _sd_spi_transfer;
    ctx->hal.cs_low    = _sd_cs_low;
    ctx->hal.cs_high   = _sd_cs_high;

    /* O delay_ms deve ter sido configurado antes (via sd_begin ou manualmente) */

    return sd_init_retry(ctx);
}

SD_Status sd_begin(SD_Context *ctx, void (*delay_func)(uint32_t ms))
{
    if (ctx == NULL || delay_func == NULL)
        return SD_ERR_READY;

    /* Injeta apenas o delay (o resto é feito pelo quick_init) */
```

```
    ctx->hal.delay_ms = delay_func;

    return sd_quick_init(ctx);
}

/* ================================================================
   STATUS
   ================================================================ */

/**
 * @brief Obtém o status atual do cartão SD (comando CMD13)
 *
 * Retorna o registrador de status R1 + R2 combinados.
 * Útil para verificar erros antes de operações de escrita longas.
 *
 * @param ctx Ponteiro para o contexto da biblioteca
 * @return Status combinado (R1 << 8 | R2). 0x0000 = cartão pronto.
 */
uint16_t sd_get_status(SD_Context *ctx)
{
    uint8_t r1, r2;

    r1 = sd_cmd_send(ctx, 13, 0, 0xAF);
    r2 = ctx->hal.spi_tx_rx(0xFF);

    ctx->hal.cs_high();

    return ((uint16_t)r1 << 8) | r2;
}

/**
 * @brief Lê um único bloco de 512 bytes do cartão (LBA).
 *
 * @param ctx Ponteiro para o contexto da biblioteca.
 * @param block Número do bloco físico (LBA) a ser lido.
 * @param buffer Ponteiro para o buffer de destino (mínimo 512 bytes).
 * @return SD_Status SD_SUCCESS ou erro de timeout/comunicação.
 */
SD_Status sd_blk_read(SD_Context *ctx, uint32_t block, uint8_t *buffer)
{
    uint32_t addr = (ctx->card_type == 1) ? (block << 9) : block;  // SDSC usa byte addressing
    uint32_t timeout = 50000;
    uint8_t token;

    if (sd_cmd_send(ctx, 17, addr, 0xFF) != 0x00) {
        ctx->hal.cs_high();
        return SD_ERR_READY;
    }

    /* Aguarda token de dados (0xFE) */
    do {
        token = ctx->hal.spi_tx_rx(0xFF);
    } while (token != 0xFE && --timeout);

    if (timeout == 0) {
        ctx->hal.cs_high();
        return SD_ERR_TIMEOUT;
    }

    /* Lê os 512 bytes */
    for (uint16_t i = 0; i < 512; i++) {
        buffer[i] = ctx->hal.spi_tx_rx(0xFF);
    }

    /* Descarta CRC (2 bytes) */
    ctx->hal.spi_tx_rx(0xFF);
```

```
    ctx->hal.spi_tx_rx(0xFF);

    ctx->hal.cs_high();
    return SD_SUCCESS;
}

/**
 * @brief Lê múltiplos blocos sequenciais
 *
 * @param ctx Ponteiro para o contexto da biblioteca.
 * @param start_block Bloco inicial
 * @param num_blocks  Quantidade de blocos a ler
 * @param buffer      Buffer de destino (deve ter num_blocks * 512 bytes)
 * @return SD_Status  Status da operação de leitura múltipla.
 */
SD_Status sd_blk_read_multi(SD_Context *ctx, uint32_t start_block, uint16_t num_blocks, uint8_t *buffer)
{
    for (uint16_t i = 0; i < num_blocks; i++) {
        SD_Status s = sd_blk_read(ctx, start_block + i, &buffer[i * 512]);
        if (s != SD_SUCCESS)
            return s;
    }
    return SD_SUCCESS;
}

/**
 * @brief Escreve um único bloco de 512 bytes
 *
 * Inclui verificação de status do cartão antes da escrita e timeout seguro.
 */
SD_Status sd_blk_write(SD_Context *ctx, uint32_t block, const uint8_t *buffer)
{
    uint32_t addr = (ctx->card_type == 1) ? (block << 9) : block;

    /* Verifica se o cartão está pronto antes de escrita longa */
    if (sd_get_status(ctx) != 0x0000) {
        ctx->hal.cs_high();
        return SD_ERR_READY;
    }

    if (sd_cmd_send(ctx, 24, addr, 0xFF) != 0x00) {
        ctx->hal.cs_high();
        return SD_ERR_WRITE;
    }

    ctx->hal.spi_tx_rx(0xFE);                   // Token de início de dados

    for (uint16_t i = 0; i < 512; i++) {
        ctx->hal.spi_tx_rx(buffer[i]);
    }

    ctx->hal.spi_tx_rx(0xFF);                   // CRC dummy
    ctx->hal.spi_tx_rx(0xFF);

    /* Verifica resposta de dados */
    if ((ctx->hal.spi_tx_rx(0xFF) & 0x1F) != 0x05) {
        ctx->hal.cs_high();
        return SD_ERR_WRITE;
    }

    /* Aguarda o cartão terminar de escrever (timeout seguro) */
    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) {
        ctx->hal.cs_high();
        return SD_ERR_TIMEOUT;
    }
```

```
    ctx->hal.cs_high();
    return SD_SUCCESS;
}

/**
 * @brief Escreve múltiplos blocos de 512 bytes usando o comando CMD25.
 *
 * Utiliza a otimização ACMD23 para pré-apagar blocos no Flash, melhorando a performance.
 * Implementa o protocolo de tokens: 0xFC para início de bloco e 0xFD para
 * sinalizar o fim da transmissão (Stop Tran).
 *
 * @param ctx Ponteiro para o contexto da biblioteca.
 * @param start_block Bloco físico inicial (LBA).
 * @param num_blocks Quantidade de blocos sequenciais a gravar.
 * @param buffer Ponteiro para os dados (se NULL, usa o buffer interno do contexto).
 * @return SD_Status Status da operação de escrita múltipla.
 */
SD_Status sd_blk_write_multi(SD_Context *ctx, uint32_t start_block, uint16_t num_blocks, const uint8_t *buffer)
{
    uint32_t addr = (ctx->card_type == 1) ? (start_block << 9) : start_block;

    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) return SD_ERR_TIMEOUT;

    // 1. Otimização: ACMD23 (SET_WR_BLK_ERASE_COUNT)
    // Informa ao cartão o número de blocos para acelerar a gravação no Flash
    sd_cmd_send(ctx, 55, 0, 0xFF); // APP_CMD
    sd_cmd_send(ctx, 23, num_blocks, 0xFF);

    // 2. Enviar CMD25 (WRITE_MULTIPLE_BLOCK)
    if (sd_cmd_send(ctx, 25, addr, 0xFF) != 0x00) {
        ctx->hal.cs_high();
        return SD_ERR_WRITE;
    }

    // 3. Loop de transmissão de dados
    for (uint16_t i = 0; i < num_blocks; i++) {
        const uint8_t *ptr = (buffer == NULL) ? ctx->buffer : &buffer[i * 512];

        if (sd_spi_wait_ready(ctx) != SD_SUCCESS) break;

        // Token de início de dados para CMD25 (0xFC)
        ctx->hal.spi_tx_rx(0xFC);

        // Transmissão do bloco de 512 bytes
        for (uint16_t b = 0; b < 512; b++) {
            ctx->hal.spi_tx_rx(ptr[b]);
        }

        // Enviar 2 bytes de CRC (dummy)
        ctx->hal.spi_tx_rx(0xFF);
        ctx->hal.spi_tx_rx(0xFF);

        // Verificar resposta de dados (Data Response)
        // 0x05 = Dados aceitos
        if ((ctx->hal.spi_tx_rx(0xFF) & 0x1F) != 0x05) {
            ctx->hal.cs_high();
            return SD_ERR_WRITE;
        }
    }

    // 4. Finalização: Enviar Stop Tran Token (0xFD)
    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) return SD_ERR_TIMEOUT;
    ctx->hal.spi_tx_rx(0xFD);

    // Aguarda o cartão terminar de processar o último bloco
    if (sd_spi_wait_ready(ctx) != SD_SUCCESS) {
```

```
        ctx->hal.cs_high();
        return SD_ERR_TIMEOUT;
    }

    ctx->hal.cs_high();
    ctx->hal.spi_tx_rx(0xFF); // Clock de sincronização final
    return SD_SUCCESS;
}

SD_Status sd_fat_mount(SD_Context *ctx, FAT32_Info *fs)
{
    uint32_t partition_lba;
    uint32_t fat_size;
    uint16_t reserved_sectors;
    uint8_t num_fats;
    uint16_t fsinfo_sector_offset;

    /* 1. LER MBR (Setor 0 Absoluto) */
    if (sd_blk_read(ctx, 0, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_READY;
    }

    /* Verifica assinatura de boot 0x55AA no MBR */
    if (ctx->buffer[510] != 0x55 || ctx->buffer[511] != 0xAA) {
        return SD_ERR_MBR;
    }

    /* Localiza o LBA da primeira partição primária (Offset 454 no MBR) */
    partition_lba = *((uint32_t *)&ctx->buffer[454]);

    /* 2. LER BOOT RECORD (Primeiro setor da partição) */
    if (sd_blk_read(ctx, partition_lba, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_READY;
    }

    /* Verifica assinatura de boot na partição (Volume Boot Record) */
    if (ctx->buffer[510] != 0x55 || ctx->buffer[511] != 0xAA) {
        return SD_ERR_BOOT;
    }

    /* 3. EXTRAIR PARÂMETROS DO BPB (BIOS Parameter Block) */

    /* Setores por Cluster (Offset 13) */
    fs->sct_per_clus = ctx->buffer[13];

    /* Setores Reservados (Offset 14) */
    reserved_sectors = *((uint16_t *)&ctx->buffer[14]);

    /* Número de tabelas FAT (Offset 16) - Quase sempre 2 */
    num_fats = ctx->buffer[16];

    /* Tamanho de uma FAT em setores (Offset 36 para FAT32) */
    fat_size = *((uint32_t *)&ctx->buffer[36]);

    /* Cluster inicial do Diretório Raiz (Offset 44) - Padrão é 2 */
    fs->root_cluster = *((uint32_t *)&ctx->buffer[44]);

    /* Localização do setor FSInfo (Offset 48 - 2 bytes) */
    fsinfo_sector_offset = *((uint16_t *)&ctx->buffer[48]);

    /* 4. CALCULAR ENDEREÇOS LBA FÍSICOS */

    fs->fat_size = fat_size;

    fs->num_fats = num_fats;
```

```
    /* LBA onde começa a primeira FAT */
    fs->fat_lba = partition_lba + reserved_sectors;

    /* LBA onde começam os dados (Clusters) */
    fs->data_start = fs->fat_lba + (num_fats * fat_size);

    /* === NOVO: LBA do diretório raiz (root) === */
    fs->root_dir_lba = fs->data_start + (fs->root_cluster - 2) * fs->sct_per_clus;

    fs->fsinfo_lba = partition_lba + fsinfo_sector_offset;

    /* 5. INICIALIZAÇÃO DE CACHES (CRÍTICO PARA O(1)) */

    /* Inicializa o hint para próxima alocação.
       Pode ser extraído do FSInfo no futuro para maior velocidade. */
    fs->next_free_cluster = 2;

    /* Invalida o cache de cluster (força busca na primeira escrita) */
    fs->last_cluster = 0xFFFFFFFF;

    /* Invalida o cache de setor FAT (força leitura no primeiro acesso) */
    fs->fat_cache_lba = 0xFFFFFFFF;

    fs->last_dir_lba = fs->root_dir_lba;
    fs->last_dir_offset = 0;

    return SD_SUCCESS;
}

/**
 * @brief Atualiza uma entrada na Tabela de Alocação de Arquivos (FAT).
 *
 * Escreve o valor do próximo cluster (ou marcador EOC) em ambas as cópias da FAT
 * (FAT1 e FAT2) para manter a redundância e integridade dos dados. Utiliza o
 * cache interno para evitar escritas desnecessárias se o setor já estiver carregado.
 *
 * @param ctx   Ponteiro para o contexto da biblioteca.
 * @param fs    Ponteiro para a estrutura FAT32 montada.
 * @param cluster O número do cluster cuja entrada será atualizada.
 * @param value   O valor de 32 bits a ser gravado (próximo cluster ou 0x0FFFFFFF).
 * @return SD_Status SD_SUCCESS em caso de escrita bem-sucedida.
 */
SD_Status sd_fat_write_entry(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster, uint32_t value)
{
    // Validação de segurança: Não permite escrever fora dos limites da FAT
    uint32_t total_clusters = fs->fat_size * 128;
    if (cluster >= total_clusters) return SD_ERR_WRITE;

    uint32_t fat_bases[2] = { fs->fat_lba, fs->fat_lba + fs->fat_size };

    // Otimização: Só sincroniza a FAT2 se ela de fato existir (num_fats > 1)
    // Nota: num_fats deve ser capturado no mount. Assumiremos 2 para conformidade FAT32.
    for (uint8_t f = 0; f < fs->num_fats; f++)
    {
        uint32_t sector = fat_bases[f] + (cluster / 128);
        uint16_t offset = (cluster % 128) * 4;

        // Leitura com verificação de erro
        if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS)
            return SD_ERR_READY;

        // Preserva os 4 bits superiores (reservados no FAT32)
        uint32_t entry = *((uint32_t *)&ctx->buffer[offset]);
        entry &= 0xF0000000;
        entry |= (value & 0x0FFFFFFF);
```

```
        *((uint32_t *)&ctx->buffer[offset]) = entry;

        // Escrita física
        if (sd_blk_write(ctx, sector, ctx->buffer) != SD_SUCCESS)
            return SD_ERR_WRITE;
    }
    return SD_SUCCESS;
}

/**
 * @brief Libera uma cadeia de clusters na tabela FAT, marcando-os como disponíveis.
 *
 * Percorre a lista encadeada a partir do cluster inicial e define cada entrada
 * como 0x00000000. Esta operação é essencial para que o espaço de arquivos
 * deletados possa ser reutilizado por novas gravações.
 *
 * @param ctx Ponteiro para o contexto da biblioteca SD.
 * @param fs Ponteiro para o sistema FAT32 montado.
 * @param start_cluster O primeiro cluster da sequência a ser liberada.
 * @return SD_Status SD_SUCCESS ou erro de escrita na tabela FAT.
 */
SD_Status sd_fat_free_chain(SD_Context *ctx, FAT32_Info *fs, uint32_t start_cluster) {
    uint32_t current = start_cluster;
    uint32_t next;
    uint32_t last_sector = 0xFFFFFFFF;

    while (current >= 2 && current < 0x0FFFFFF8) {
        uint32_t fat_sector = fs->fat_lba + (current / 128);

        if (fat_sector != last_sector) {
            if (last_sector != 0xFFFFFFFF) {
                if (sd_blk_write(ctx, last_sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;
            }
            if (sd_blk_read(ctx, fat_sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;
            last_sector = fat_sector;
        }

        next = ((uint32_t *)ctx->buffer)[current % 128] & 0x0FFFFFFF;
        ((uint32_t *)ctx->buffer)[current % 128] = 0x00000000;
        current = next;
    }

    if (last_sector != 0xFFFFFFFF) {
        return sd_blk_write(ctx, last_sector, ctx->buffer);
    }
    return SD_SUCCESS;
}

/**
 * @brief Aloca um cluster livre na FAT32 utilizando busca circular otimizada.
 *
 * @param ctx Ponteiro para o contexto do SD.
 * @param fs Ponteiro para a estrutura FAT32 montada.
 * @param cluster_out Ponteiro para retornar o endereço do cluster alocado.
 * @return SD_Status SD_SUCCESS ou SD_ERR_DISK_FULL se não houver espaço.
 */
SD_Status sd_fat_alloc_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t *cluster_out)
{
    uint32_t sector;
    uint16_t offset;
    uint32_t total_clusters = fs->fat_size * 128;
    uint32_t start_search = fs->next_free_cluster;

    // Garante que o hint seja válido
    if (start_search < 2 || start_search >= total_clusters) start_search = 2;
```

```
    uint32_t cl = start_search;
    uint8_t wrapped = 0;

    while (1) {
        sector = fs->fat_lba + (cl / 128);
        offset = (cl % 128) * 4;

        if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

        uint32_t entry = *((uint32_t *)&ctx->buffer[offset]) & 0x0FFFFFFF;

        if (entry == 0) {
            // Encontrou livre! Marca como EOC (Fim de Cadeia)
            if (sd_fat_write_entry(ctx, fs, cl, 0x0FFFFFFF) != SD_SUCCESS) return SD_ERR_WRITE;

            *cluster_out = cl;
            fs->next_free_cluster = cl + 1; // Atualiza o "hint" para a próxima vez
            return SD_SUCCESS;
        }

        cl++;

        // Se chegou ao fim da FAT, volta para o começo (busca circular)
        if (cl >= total_clusters) {
            if (wrapped) break; // Já deu uma volta completa e está tudo cheio
            cl = 2;
            wrapped = 1;
        }

        // Se deu a volta e alcançou o ponto de partida original
        if (wrapped && cl >= start_search) break;
    }

    return SD_ERR_DISK_FULL;
}

/**
 * @brief Aloca um novo cluster e o encadeia ao final de uma sequência existente.
 *
 * @param ctx             Ponteiro para o contexto.
 * @param fs              Ponteiro para a FAT.
 * @param current_cluster  Cluster atual da cadeia.
 * @param new_cluster      Ponteiro para receber o novo cluster alocado.
 * @return SD_Status Status da alocação e escrita na FAT.
 */
SD_Status sd_fat_chain_new_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t current_cluster, uint32_t
*new_cluster)
{
    /* 1. Aloca novo cluster usando a função da pág 11 */
    if (sd_fat_alloc_cluster(ctx, fs, new_cluster) != SD_SUCCESS)
        return SD_ERR_DISK_FULL;

    /* 2. Atualiza FAT[current] -> new_cluster (pág 10) */
    if (sd_fat_write_entry(ctx, fs, current_cluster, *new_cluster) != SD_SUCCESS)
        return SD_ERR_WRITE;

    return SD_SUCCESS;
}

/**
 * @brief Obtém próximo cluster na FAT
 *
 * @param ctx      Ponteiro para o contexto.
 * @param fs       Ponteiro para a FAT.
 * @param cluster  Cluster para consultar.
 *
```

```
 * @return uint32_t Próximo cluster ou erro
 */
uint32_t sd_fat_next_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster)
{
    uint32_t sector = fs->fat_lba + (cluster / 128);
    uint16_t offset = (cluster % 128) * 4;

    // Se o setor já estiver no buffer da ctx (cache), não lê do SD de novo
    if (fs->fat_cache_lba != sector) {
        if (sd_blk_read(ctx, sector, ctx->buffer) != SD_SUCCESS) {
            fs->fat_cache_lba = 0xFFFFFFFF; // Invalida em erro
            return 0xFFFFFFFF;
        }
        fs->fat_cache_lba = sector;
    }

    // Lê a entrada da FAT diretamente do buffer
    uint32_t next = *((uint32_t *)&ctx->buffer[offset]);
    return (next & 0x0FFFFFFF);
}

/**
 * @brief Atualiza o setor FSInfo do FAT32.
 *
 * Escreve o número de clusters livres e o último cluster alocado para que
 * o sistema operacional não precise reescaneia o disco todo ao ser montado.
 *
 * @param ctx Ponteiro para o contexto SD.
 * @param fs Ponteiro para o sistema FAT32 montado.
 * @param free_clusters Qtd de clusters livres (use 0xFFFFFFFF se desconhecido).
 * @param last_alloc Último cluster alocado (ajuda na velocidade de busca).
 * @return SD_Status Status da gravação.
 */
SD_Status sd_fat_update_fsinfo(SD_Context *ctx, FAT32_Info *fs, uint32_t free_clusters, uint32_t last_alloc)
{
    if (sd_blk_read(ctx, fs->fsinfo_lba, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_READY;
    }

    // Validação de Assinaturas (Offset 0, 484 e 510)
    // Essencial para evitar sobrescrever setores de dados por erro de LBA
    uint32_t lead_sig = *((uint32_t *)&ctx->buffer[0]);
    uint32_t struc_sig = *((uint32_t *)&ctx->buffer[484]);
    uint16_t trail_sig = *((uint16_t *)&ctx->buffer[510]);

    if (lead_sig != 0x41615252 || struc_sig != 0x61417272 || trail_sig != 0xAA55) {
        return SD_ERR_BOOT; // Estrutura FSInfo inválida ou setor errado
    }

    // Atualiza apenas se os valores forem lógicos
    if (free_clusters != 0xFFFFFFFF) {
        *((uint32_t *)&ctx->buffer[488]) = free_clusters;
    }

    *((uint32_t *)&ctx->buffer[492]) = last_alloc;

    if (sd_blk_write(ctx, fs->fsinfo_lba, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_WRITE;
    }
    return SD_SUCCESS;
}

/* ================================================================
   CAMADA DE ARQUIVO
   ================================================================ */
```

```
/**
 * @brief Localiza a entrada de um arquivo no diretório raiz.
 *
 * Percorre a cadeia de clusters do diretório raiz comparando o nome e a extensão.
 * Ignora entradas marcadas como deletadas (0xE5) e interrompe a busca ao encontrar
 * o marcador de fim de diretório (0x00).
 *
 * @param ctx        Ponteiro para o contexto da biblioteca.
 * @param fs         Ponteiro para a estrutura FAT32 montada.
 * @param file       Ponteiro para a estrutura SD_FileName (nome formatado 8.3).
 * @param entry_out  Retorna o deslocamento (offset) da entrada dentro do setor.
 * @param lba_out    Retorna o endereço físico (LBA) do setor onde o arquivo foi achado.
 * @return SD_Status SD_SUCCESS se encontrado, SD_ERR_NOT_FOUND caso contrário.
 */
SD_Status sd_find_file_entry(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint16_t *entry_out,
uint32_t *lba_out) {
    uint32_t cluster = fs->root_cluster;
    while (cluster < 0x0FFFFFF8 && cluster >= 2) {
        for (uint8_t s = 0; s < fs->sct_per_clus; s++) {
            uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus + s;
            if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

            for (uint16_t i = 0; i < 512; i += 32) {
                // Se encontrar 0x00, significa que não há mais arquivos NESTE cluster.
                // Mas ainda pode haver arquivos no PRÓXIMO cluster da corrente.
                if (ctx->buffer[i] == 0x00) break;

                if (ctx->buffer[i] == 0xE5) continue;
                if (!memcmp(&ctx->buffer[i], file->name, 8) && !memcmp(&ctx->buffer[i+8], file->ext, 3)) {
                    *entry_out = i;
                    *lba_out = lba;
                    return SD_SUCCESS;
                }
            }
        }
        // Pula para o próximo cluster da FAT (Pág. 12/21)
        if (sd_fat_get_next_cluster(ctx, fs, cluster, &cluster) != SD_SUCCESS) break;
    }
    return SD_ERR_NOT_FOUND;
}

SD_Status sd_file_exists(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    // Substitui o loop for fixo de 512 bytes pela busca dinâmica
    return sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba);
}

SD_Status sd_file_get_size(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t *size)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    // Localiza a entrada em qualquer lugar da cadeia de clusters do root
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
        return SD_ERR_NOT_FOUND;
    }

    // O buffer já contém o setor correto graças à sd_find_file_entry
    *size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    return SD_SUCCESS;
}
```

```
SD_Status sd_file_read(SD_Context *ctx, FAT32_Info *fs,
                       const SD_FileName *file,
                       uint8_t *buffer,
                       uint32_t buffer_size)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    uint32_t cluster;
    uint32_t file_size;
    uint32_t bytes_read = 0;

    /* 1. Localiza arquivo */
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
        return SD_ERR_NOT_FOUND;

    /* 2. Cluster inicial */
    cluster =
        ((uint32_t)ctx->buffer[entry_offset + 26]) |
        ((uint32_t)ctx->buffer[entry_offset + 27] << 8) |
        ((uint32_t)ctx->buffer[entry_offset + 20] << 16) |
        ((uint32_t)ctx->buffer[entry_offset + 21] << 24);

    /* 3. Tamanho do arquivo */
    file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    /* 4. Define limite real de leitura (LEITURA PARCIAL SEGURA) */
    uint32_t max_to_read = (file_size < buffer_size) ? file_size : buffer_size;

    /* 5. Percorre cadeia de clusters */
    while (cluster < 0x0FFFFFF8 && cluster >= 2)
    {
        uint32_t lba = fs->data_start +
                       (cluster - 2) * fs->sct_per_clus;

        for (uint8_t s = 0; s < fs->sct_per_clus; s++)
        {
            if (bytes_read >= max_to_read)
                return SD_SUCCESS;

            if (sd_blk_read(ctx, lba + s, ctx->buffer) != SD_SUCCESS)
                return SD_ERR_READY;

            uint16_t remaining = max_to_read - bytes_read;
            uint16_t chunk = (remaining > 512) ? 512 : remaining;

            memcpy(&buffer[bytes_read], ctx->buffer, chunk);
            bytes_read += chunk;
        }

        cluster = sd_fat_next_cluster(ctx, fs, cluster);

        if (cluster == 0xFFFFFFFF)
            return SD_ERR_READY;
    }

    return SD_SUCCESS;
}

SD_Status sd_file_read_stream(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, sd_stream_callback_t
callback)
{
    uint16_t entry_offset;
    uint32_t entry_lba;
    uint32_t cluster;
    uint32_t file_size;
```

```
    uint32_t bytes_read = 0;

    // 1. Localizar arquivo
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
        return SD_ERR_NOT_FOUND;

    // 2. Extração correta do cluster inicial de 32 bits (Bytes 20 e 26)
    uint32_t cluster_high = *((uint16_t *)&ctx->buffer[entry_offset + 20]);
    uint32_t cluster_low = *((uint16_t *)&ctx->buffer[entry_offset + 26]);
    cluster = (cluster_high << 16) | cluster_low;

    file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    // Se o arquivo existir mas estiver vazio, termina com sucesso
    if (file_size == 0 || cluster < 2) return SD_SUCCESS;

    // 3. Loop de leitura por cadeia de clusters
    while (cluster < 0x0FFFFFF8 && cluster >= 2)
    {
        uint32_t lba_base = fs->data_start + (cluster - 2) * fs->sct_per_clus;

        for (uint8_t s = 0; s < fs->sct_per_clus; s++)
        {
            if (bytes_read >= file_size) return SD_SUCCESS;

            if (sd_blk_read(ctx, lba_base + s, ctx->buffer) != SD_SUCCESS)
                return SD_ERR_READY;

            uint16_t remaining = file_size - bytes_read;
            uint16_t chunk = (remaining > 512) ? 512 : remaining;

            callback(ctx->buffer, chunk);
            bytes_read += chunk;
        }

        // Navega para o proximo cluster usando sua funcao da pag 21
        if (sd_fat_get_next_cluster(ctx, fs, cluster, &cluster) != SD_SUCCESS)
            return SD_ERR_READY;
    }

    return SD_SUCCESS;
}

SD_Status sd_file_update_size(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t new_size)
{
    uint16_t entry_offset;
    uint32_t entry_lba;

    // 1. Localiza a posição física da entrada
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
        return SD_ERR_NOT_FOUND;
    }

    // 2. Atualiza o tamanho no buffer (o buffer foi preenchido pela sd_find_file_entry)
    *((uint32_t *)&ctx->buffer[entry_offset + 28]) = new_size;

    // 3. Grava o setor de volta no LBA correto (não mais fixo no root_dir_lba)
    if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) {
        return SD_ERR_WRITE;
    }

    return SD_SUCCESS;
}

SD_Status sd_file_write_stream(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, sd_write_callback_t
callback)
```

```
{
    uint16_t entry_offset;
    uint32_t entry_lba;
    uint32_t cluster;
    uint32_t file_size;
    uint32_t bytes_written = 0;

    // 1. Localiza entrada do arquivo
    if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
        return SD_ERR_NOT_FOUND;

    // 2. Extração robusta do cluster inicial (High + Low Word)
    uint32_t cluster_high = *((uint16_t *)&ctx->buffer[entry_offset + 20]);
    uint32_t cluster_low = *((uint16_t *)&ctx->buffer[entry_offset + 26]);
    cluster = (cluster_high << 16) | cluster_low;

    // 3. Tamanho atual do arquivo
    file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

    // Se o arquivo estiver vazio, não há nada para sobrescrever nesta função (use append)
    if (file_size == 0 || cluster < 2) return SD_SUCCESS;

    // 4. Loop de escrita por cadeia de clusters
    while (cluster < 0x0FFFFFF8 && cluster >= 2)
    {
        uint32_t lba_base = fs->data_start + (cluster - 2) * fs->sct_per_clus;

        for (uint8_t s = 0; s < fs->sct_per_clus; s++)
        {
            if (bytes_written >= file_size) return SD_SUCCESS;

            uint32_t lba = lba_base + s;
            uint16_t remaining = file_size - bytes_written;
            uint16_t chunk = (remaining > 512) ? 512 : remaining;

            // Limpa buffer e solicita dados do usuário via callback
            memset(ctx->buffer, 0x00, 512);
            uint16_t produced = callback(ctx->buffer, chunk);

            if (produced == 0) return SD_SUCCESS; // Usuário encerrou a transmissão
            if (produced > chunk) produced = chunk; // Proteção contra overflow

            // Escrita física no bloco
            if (sd_blk_write(ctx, lba, ctx->buffer) != SD_SUCCESS)
                return SD_ERR_WRITE;

            bytes_written += produced;
        }

        // Navega para o próximo cluster da corrente
        if (sd_fat_get_next_cluster(ctx, fs, cluster, &cluster) != SD_SUCCESS)
            return SD_ERR_READY;
    }

    return SD_SUCCESS;
}

SD_Status sd_file_append_stream(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, sd_stream_reader_t
reader)
{
    uint16_t entry_offset;
    uint32_t entry_lba;
    uint32_t cluster;
    uint32_t file_size;
    uint32_t sector_offset;
    uint16_t offset;
```

```
uint16_t bytes_produced; // Variável para armazenar o retorno real do callback


// 1. Localizar entrada do arquivo para obter estado atual
if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS)
    return SD_ERR_NOT_FOUND;

// O buffer ja contem o setor da entrada gracas a sd_find_file_entry
file_size = *((uint32_t *)&ctx->buffer[entry_offset + 28]);

// Extração correta do cluster inicial (High + Low)
uint32_t first_cl = ((uint32_t)*((uint16_t *)&ctx->buffer[entry_offset + 20]) << 16) |
                    (*((uint16_t *)&ctx->buffer[entry_offset + 26]));

// 2. Se o arquivo estiver vazio (cluster < 2), aloca o PRIMEIRO cluster agora
if (first_cl < 2) {
    if (sd_fat_alloc_cluster(ctx, fs, &first_cl) != SD_SUCCESS) return SD_ERR_DISK_FULL;

    // Marca como fim de corrente na FAT (EOC)
    sd_fat_write_entry(ctx, fs, first_cl, 0x0FFFFFFF);

    // Atualiza a entrada de diretorio física com o novo cluster inicial
    sd_blk_read(ctx, entry_lba, ctx->buffer);
    *((uint16_t *)&ctx->buffer[entry_offset + 20]) = (uint16_t)(first_cl >> 16);
    *((uint16_t *)&ctx->buffer[entry_offset + 26]) = (uint16_t)(first_cl & 0xFFFF);
    sd_blk_write(ctx, entry_lba, ctx->buffer);

    cluster = first_cl;
} else {
    // Se ja tem dados, navega ate o ultimo cluster da corrente
    cluster = first_cl;
    while (1) {
        uint32_t next;
        if (sd_fat_get_next_cluster(ctx, fs, cluster, &next) != SD_SUCCESS) break;
        if (next >= 0x0FFFFFF8) break;
        cluster = next;
    }
}

// Calcula posição de escrita dentro do cluster
offset = file_size % 512;
sector_offset = (file_size / 512) % fs->sct_per_clus;

// 3. Loop de escrita eficiente
const uint8_t *data;
while ((bytes_produced = reader(&data)) > 0)
{
    uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus + sector_offset;

    if (offset == 0 && bytes_produced == 512) {
        // Escrita direta de bloco (Otimizado para blocos cheios)
        if (sd_blk_write(ctx, lba, data) != SD_SUCCESS) return SD_ERR_WRITE;
    } else {
        // Read-Modify-Write para manter dados existentes e adicionar novos
        if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

        // Garante que não ultrapasse o limite do setor atual
        uint16_t space_left = 512 - offset;
        uint16_t to_copy = (bytes_produced < space_left) ? bytes_produced : space_left;

        memcpy(&ctx->buffer[offset], data, to_copy);
        if (sd_blk_write(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;

        // Se a string for maior que o espaço no setor (raro em logs curtos)
        // bytes_produced precisaria ser tratado em loop, mas para logs de sensores
        // o código acima é seguro.
```

```
        }

        // ATUALIZAÇÃO CRÍTICA: Soma apenas os bytes realmente enviados
        file_size += bytes_produced;

        // Recalcula ponteiros para a próxima iteração
        offset = (uint16_t)(file_size % 512);
        sector_offset = (uint32_t)((file_size / 512) % fs->sct_per_clus);

        // Se o cluster encheu, aloca o próximo
        if (offset == 0 && sector_offset == 0) {
            uint32_t next;
            if (sd_fat_chain_new_cluster(ctx, fs, cluster, &next) != SD_SUCCESS)
                return SD_ERR_DISK_FULL;
            cluster = next;
        }
    }

    // 4. Atualiza o tamanho final no diretorio
    return sd_file_update_size(ctx, fs, file, file_size);
}

SD_Status sd_file_create(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file_name) {
    uint32_t cluster = fs->root_cluster;
    uint32_t last_cluster = cluster;
    uint32_t free_lba = 0;
    uint16_t free_off = 0;
    uint8_t found_space = 0;

    // Percorre o diretório UMA ÚNICA VEZ
    while (cluster < 0x0FFFFFF8 && cluster >= 2) {
        for (uint8_t s = 0; s < fs->sct_per_clus; s++) {
            uint32_t lba = fs->data_start + (cluster - 2) * fs->sct_per_clus + s;
            if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

            for (uint16_t off = 0; off < 512; off += 32) {
                // Se encontrar o nome, o arquivo JÁ EXISTE
                if (ctx->buffer[off] != 0x00 && ctx->buffer[off] != 0xE5) {
                    if (!memcmp(&ctx->buffer[off], file_name->name, 8) &&
                        !memcmp(&ctx->buffer[off+8], file_name->ext, 3)) {
                        return SD_SUCCESS;
                    }
                }
                // Se encontrar espaço livre e ainda não salvamos um local, guarda a coordenada
                else if (!found_space) {
                    free_lba = lba;
                    free_off = off;
                    found_space = 1;
                }

                // Fim absoluto das entradas deste cluster
                if (ctx->buffer[off] == 0x00) break;
            }
        }
        last_cluster = cluster;
        if (sd_fat_get_next_cluster(ctx, fs, cluster, &cluster) != SD_SUCCESS) break;
    }

    // Se achou um espaço vazio durante a busca acima, usa ele agora
    if (found_space) {
        if (free_lba != 0) {
            // Recarrega o setor para não perder outras entradas
            sd_blk_read(ctx, free_lba, ctx->buffer);

            // Limpa os 32 bytes da entrada (Zera tamanhos e clusters iniciais)
            memset(&ctx->buffer[free_off], 0, 32);
```

```
            // Preenche Nome e Extensão
            memcpy(&ctx->buffer[free_off], file_name->name, 11);

            // Atributo: Arquivo (Archive)
            ctx->buffer[free_off + 11] = 0x20;

            // --- Gravação de Data e Hora ---
            uint16_t f_date = _sd_make_fat_date(g_sd_datetime.year, g_sd_datetime.month, g_sd_datetime.day);
            uint16_t f_time = _sd_make_fat_time(g_sd_datetime.hour, g_sd_datetime.minute, g_sd_datetime.second);

            // Grava Data/Hora de Criação (Offsets 14 e 16)
            *((uint16_t *)&ctx->buffer[free_off + 14]) = f_time;
            *((uint16_t *)&ctx->buffer[free_off + 16]) = f_date;

            // Grava Data de último acesso (Offset 18)
            *((uint16_t *)&ctx->buffer[free_off + 18]) = f_date;

            // Grava Data/Hora de modificação (Offsets 22 e 24)
            *((uint16_t *)&ctx->buffer[free_off + 22]) = f_time;
            *((uint16_t *)&ctx->buffer[free_off + 24]) = f_date;

            return sd_blk_write(ctx, free_lba, ctx->buffer);
        }
    }

    // Se não achou espaço em nenhum cluster existente, expande o diretório
    if (sd_expand_directory(ctx, fs, last_cluster) == SD_SUCCESS) {
        return sd_file_create(ctx, fs, file_name);
    }

    return SD_ERR_DISK_FULL;
}

/**
 * @brief Obtém o próximo cluster na tabela FAT.
 * @param ctx          Ponteiro para o contexto.
 * @param fs           Ponteiro para a FAT.
 * @param cluster      Cluster atual.
 * @param next_cluster  Ponteiro para retornar o valor do próximo.
 * @return SD_Status Status da leitura do setor da FAT.
 */
SD_Status sd_fat_get_next_cluster(SD_Context *ctx, FAT32_Info *fs, uint32_t cluster, uint32_t *next_cluster) {
    uint32_t fat_sector = fs->fat_lba + (cluster / 128);
    if (sd_blk_read(ctx, fat_sector, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

    *next_cluster = ((uint32_t *)ctx->buffer)[cluster % 128] & 0x0FFFFFFF;
    return SD_SUCCESS;
}

SD_Status sd_file_delete(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file) {
    uint32_t current_cluster = fs->root_cluster;
    uint32_t cluster_to_free = 0;

    while (current_cluster >= 2 && current_cluster < 0x0FFFFFF8) {
        for (uint8_t sct = 0; sct < fs->sct_per_clus; sct++) {
            uint32_t lba = fs->data_start + (current_cluster - 2) * fs->sct_per_clus + sct;

            if (sd_blk_read(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_READY;

            for (uint16_t i = 0; i < 512; i += 32) {
                // Fim do diretório neste cluster
                if (ctx->buffer[i] == 0x00) return SD_ERR_NOT_FOUND;
                // Entrada já deletada, pula
                if (ctx->buffer[i] == 0xE5) continue;
```

```
            // Compara Nome (8) + Extensão (3) = 11 bytes
            if (memcmp(&ctx->buffer[i], file->name, 11) == 0) {

                // 1. EXTRAI O CLUSTER ANTES DE QUALQUER OUTRA OPERAÇÃO
                uint32_t high = *((uint16_t *)&ctx->buffer[i + 20]);
                uint32_t low  = *((uint16_t *)&ctx->buffer[i + 26]);
                cluster_to_free = (high << 16) | low;

                // 2. MARCA COMO DELETADO E GRAVA IMEDIATAMENTE
                ctx->buffer[i] = 0xE5;
                if (sd_blk_write(ctx, lba, ctx->buffer) != SD_SUCCESS) return SD_ERR_WRITE;

                // 3. LIBERA A CADEIA NA FAT (Agora o buffer pode ser sobrescrito com segurança)
                if (cluster_to_free >= 2 && cluster_to_free < 0x0FFFFFF8) {
                    return sd_fat_free_chain(ctx, fs, cluster_to_free);
                }

                return SD_SUCCESS; // Arquivo vazio deletado com sucesso
            }
        }
    }

    // Pula para o próximo cluster do diretório raiz (se houver expansão)
    if (sd_fat_get_next_cluster(ctx, fs, current_cluster, &current_cluster) != SD_SUCCESS) break;
  }

  return SD_ERR_NOT_FOUND;
}

SD_Status sd_file_sync(SD_Context *ctx, FAT32_Info *fs, const SD_FileName *file, uint32_t current_size)
{
  uint16_t entry_offset;
  uint32_t entry_lba;

  // Localiza a entrada física do arquivo
  if (sd_find_file_entry(ctx, fs, file, &entry_offset, &entry_lba) != SD_SUCCESS) {
      return SD_ERR_NOT_FOUND;
  }

  // Atualiza o tamanho e força a escrita no cartão
  // O buffer já contém os dados do setor lido pela sd_find_file_entry
  *((uint32_t *)&ctx->buffer[entry_offset + 28]) = current_size;

  if (sd_blk_write(ctx, entry_lba, ctx->buffer) != SD_SUCCESS) {
      return SD_ERR_WRITE;
  }

  // Opcional: Atualiza o FSInfo para refletir clusters usados
  return sd_fat_update_fsinfo(ctx, fs, 0xFFFFFFFF, fs->next_free_cluster);
}

SD_Status sd_set_filename(SD_FileName *file_out, const char *name_in) {
  if (!file_out || !name_in) return SD_ERR_READY;

  // 1. Limpar a estrutura com espaços (padrão FAT)
  memset(file_out->name, ' ', 8);
  memset(file_out->ext, ' ', 3);

  uint8_t i = 0;
  // 2. Copiar o nome (até achar o ponto ou fim da string)
  while (name_in[i] != '.' && name_in[i] != '\0' && i < 8) {
      char c = name_in[i];
      if (c >= 'a' && c <= 'z') c -= 32; // Converter para maiúscula
      file_out->name[i] = c;
      i++;
  }
```

```
    // 3. Pular o ponto (se existir) e ir para a extensão
    const char *dot = strchr(name_in, '.');
    if (dot) {
        dot++; // Avança após o '.'
        for (uint8_t j = 0; j < 3 && dot[j] != '\0'; j++) {
            char c = dot[j];
            if (c >= 'a' && c <= 'z') c -= 32; // Converter para maiúscula
            file_out->ext[j] = c;
        }
    }

    return SD_SUCCESS;
}


/* ================================================================
   UTILITÁRIOS
   ================================================================ */

void sd_dir_list(SD_Context *ctx, FAT32_Info *fs) {
    uint32_t current_cluster = fs->root_cluster;
    uint8_t *entry;

    printf("\n--- Listagem do Diretorio Raiz ---\n");

    while (current_cluster >= 2 && current_cluster < 0x0FFFFFF8) {
        uint32_t lba = fs->data_start + (current_cluster - 2) * fs->sct_per_clus;

        for (uint8_t sct = 0; sct < fs->sct_per_clus; sct++) {
            if (sd_blk_read(ctx, lba + sct, ctx->buffer) != SD_SUCCESS) return;

            for (uint16_t i = 0; i < 512; i += 32) {
                entry = &ctx->buffer[i];

                if (entry[0] == 0x00) return; // Fim do diretório
                if (entry[0] == 0xE5) continue; // Arquivo deletado
                if (entry[11] & 0x0F) continue; // Pula LFN (Long File Name) ou Volume Label

                // Imprime o nome (8 caracteres)
                for (uint8_t j = 0; j < 8; j++) {
                    if (entry[j] != ' ') putchar(entry[j]);
                }

                // Imprime a extensão se existir
                if (entry[8] != ' ') {
                    putchar('.');
                    for (uint8_t j = 8; j < 11; j++) putchar(entry[j]);
                }

                // Indica se é diretório
                if (entry[11] & 0x10) printf(" <DIR>");

                printf("\n");
            }
        }
        // Busca próximo cluster do diretório
        if (sd_fat_get_next_cluster(ctx, fs, current_cluster, &current_cluster) != SD_SUCCESS) break;
    }
}

uint32_t sd_get_capacity_mb(SD_Context *ctx) {
    uint8_t csd[16];
    uint32_t c_size;

    // CMD9: SEND_CSD
```

```
    if (sd_cmd_send(ctx, 9, 0, 0xFF) != 0x00) return 0;

    // Aguarda o token de dados 0xFE
    uint16_t timeout = 10000;
    while (ctx->hal.spi_tx_rx(0xFF) != 0xFE && --timeout);
    if (timeout == 0) return 0;

    // Lê os 16 bytes do CSD
    for (uint8_t i = 0; i < 16; i++) csd[i] = ctx->hal.spi_tx_rx(0xFF);

    // Descarta CRC
    ctx->hal.spi_tx_rx(0xFF); ctx->hal.spi_tx_rx(0xFF);
    ctx->hal.cs_high();

    // Cálculo para SDHC (Versão 2.0) - O mais comum hoje
    if ((csd[0] & 0xC0) == 0x40) {
        c_size = ((((uint32_t)csd[7] & 0x3F) << 16) | ((uint32_t)csd[8] << 8) | csd[9]) + 1;
        return c_size / 2; // (C_SIZE + 1) * 512KB / 1024 = MB
    }

    return 0; // Para simplificar o foco foi em cartões modernos (SDHC/SDXC)
}

/**
 * @brief Expande o diretorio alocando um novo cluster.
 *
 * @param ctx Ponteiro para o contexto do SD.
 * @param fs Ponteiro para a estrutura FAT32.
 * @param start_cluster Cluster inicial do diretorio.
 * @return SD_Status Status da operacao.
 */
static SD_Status sd_expand_directory(SD_Context *ctx, FAT32_Info *fs, uint32_t start_cluster) {
    uint32_t current = start_cluster;
    uint32_t next;
    SD_Status st;

    // 1. Navegar ate o ultimo cluster da corrente
    while (1) {
        if (sd_fat_get_next_cluster(ctx, fs, current, &next) != SD_SUCCESS) return SD_ERR_READY;
        if (next >= 0x0FFFFFF8) break;
        current = next;
    }

    // 2. Alocar novo cluster (Usando sua funcao da pag 11)
    uint32_t new_cluster;
    st = sd_fat_alloc_cluster(ctx, fs, &new_cluster);
    if (st != SD_SUCCESS) return st;

    // 3. Ligar o antigo ao novo (Pag 10)
    st = sd_fat_write_entry(ctx, fs, current, new_cluster);
    if (st != SD_SUCCESS) return st;

    // 4. Limpar o novo cluster no disco (Zerar entradas de diretorio)
    memset(ctx->buffer, 0, 512);
    uint32_t first_sector = fs->data_start + (new_cluster - 2) * fs->sct_per_clus;

    for (uint8_t i = 0; i < fs->sct_per_clus; i++) {
        st = sd_blk_write(ctx, first_sector + i, ctx->buffer); // Funcao da pag 6
        if (st != SD_SUCCESS) return st;
    }

    return SD_SUCCESS;
}

/* ================================================================
   ESTRUTURA PARA DATA E HORA
```

```
  ================================================================ */
/**
 * @brief Converte componentes de tempo para o formato FAT (16 bits).
 * Formato: hhhhhmmmmmmsssss (segundos são divididos por 2)
 */
static uint16_t _sd_make_fat_time(uint8_t h, uint8_t m, uint8_t s) {
    return (uint16_t)(((uint16_t)(h & 0x1F) << 11) | ((uint16_t)(m & 0x3F) << 5) | (s / 2 & 0x1F));
}

/**
 * @brief Converte componentes de data para o formato FAT (16 bits).
 * Formato: yyyyyyymmmmddddd (ano desde 1980)
 */
static uint16_t _sd_make_fat_date(uint16_t year, uint8_t month, uint8_t day) {
    if (year < 1980) year = 1980;
    return (uint16_t)((((uint16_t)(year - 1980) & 0x7F) << 9) | ((uint16_t)(month & 0x0F) << 5) | (day & 0x1F));
}
```